Python

Python

Python

Data

Data

- sciene

data science

Indexing

Indexing refers to accessing elements of a collection using their position or key. The position is specified by an integer value (starting from 0 in Python), while in the case of dictionaries, keys are used instead of integer indices.

Indexing in Lists and Tuples:

- Lists and tuples are ordered collections, which means their elements can be accessed using indices.
- Positive indices start from 0, and negative indices start from -1 (from the end of the list/tuple).

Python Data structures:

List, Tuple, Set, and Dictionary

In Python, data structures are used to store and organize multiple items in a single variable. The most common are lists, tuples, sets, and dictionaries. Each of these has its own characteristics and methods.

In [ ]:
```
order - defined
mutable - change
duplicate - allows
[] - square bracket, () - parenthesis, {} - curly bracket
```

1. List

A list is an ordered, mutable (changeable) collection that allows duplicate elements. Lists are created by placing elements inside square brackets [].

Characteristics:

- **Ordered:** Elements have a defined order, and the order will not change unless explicitly modified.
- **Mutable:** You can change, add, or remove elements after the list has been created.
- **Allows duplicates:** Lists can contain duplicate elements.

In [ ]:
```
name = ['Patience', "UdoChukwu", "Salau", 'Victor', 'Sumanya', 'Fessa', "Jennifer",
```

In [ ]:
```
name
```

In [ ]:
```
type(name)
```

In [ ]:
```

```

In [ ]:
```
# Example
my_list = [1, 2, 3, 4, 2, 5]
```

In [ ]:
```
my_list
```

In [ ]:
```
list2 = [1, "hello", 3.14, [2, 3], {"key": "value"}]
# Contains integer, string, float, list, and dictionary
```

In [ ]:
```
list2[-2][0]
```

In [ ]:
```
list2.append("th")
```

In [ ]:
```
list2
```

In [ ]:
```
list2.extend("77")
```

In [ ]:
```
list2.insert(1,7)
```

In [ ]:
```
list2.pop() # works with indexing
```

In [ ]:
```
list2
```

In [ ]:
```
list2.remove(7) # works with the actual item
```

In [ ]:
```
list2.reverse()
```

In [ ]:
```
list2
```

In [ ]:
```
list2.index('hello')
```

Common List Methods:

- **append(item):** Adds an item to the end of the list.
- **extend(iterable):** Adds all elements of an iterable (like another list) to the end of the list.
- **insert(index, item):** Inserts an item at a given position.
- **remove(item):** Removes the first occurrence of an item.
- **pop(index):** Removes and returns the item at the given position (default is the last item).
- **sort():** Sorts the list in ascending order.
- **reverse():** Reverses the list.
- **count(item):** Returns the number of occurrences of an item.
- **index(item):** Returns the position of the first occurrence of an item.

In [ ]:
```
m = ['Chidi', 'Patience', 'UdoChukwu', 'Salau', 'Victor', 'Sumanya', 'Fessa', 'Jenn
print(m)
```

In [ ]:
```
m.reverse()
```

In [ ]:
```
print(m)
```

In [ ]:
```
print(name)
```

In [ ]:
```
name.index('Salau')
```

In [ ]:
```
len(m)
```

In [ ]:
```
m.clear()
```

In [ ]:
```
m
```

In [ ]:
```
students.copy()
```

In [ ]:
```
t.clear()
```

In [ ]:
```
students.append('errr')
```

In [ ]:
```
students.extend('errr')
```

In [ ]:
```
print(students)
```

In [ ]:
```
students.insert(0,'urt')
```

In [ ]:
```
students.reverse()
```

In [ ]:
```
students.remove('g')
```

In [ ]:
```
students.pop(12)
```

In [ ]:
```
students.index('ololo')
```

In [ ]:
```
students.count('5')
```

In [ ]:
```
t = ['j','juik','huy','med','hio']
```

In [ ]:
```
t.sort()
```

In [ ]:
```
t
```

In [ ]:
```
# students.sort()
```

In [ ]:
```
print(students)
```

In [ ]:

In [ ]:

In [ ]:
```
my_list.  # (hit tap to see all the methods/functions in list)
```

In [ ]:
```
# Adding an item
my_list.append(6)
```

In [ ]:
```
# Inserting an item
my_list.insert(2, 9)
```

In [ ]:
```
# Sorting the list
my_list.sort()
```

In [ ]:
```
# Reversing the list
my_list.reverse()
```

In [ ]:
```
# Count occurrences of an item
my_list.count(5)
```

In [ ]:
```
# Removing an item
my_list.remove(2)
```

In [ ]:
```
# removes the last item in a list, if no index is specified
list2.pop()
```

2. Tuple

A tuple is an ordered, immutable (unchangeable) collection that allows duplicate elements. Tuples are created by placing elements inside parentheses ().

Characteristics:

- **Ordered:** Elements have a specific order, and this order will not change.
- **Immutable:** Once a tuple is created, you cannot change its elements.
- **Allows duplicates:** Tuples can contain duplicate elements.

In [ ]:
```
names = ('Chidi', 'Patience', 'UdoChukwu', 'Salau', 'Victor', 'Sumanya', 'Fessa', (
```

In [ ]:
```
print(names)
```

In [ ]:
```
names.index('Victor')
```

In [ ]:
```
type(names)
```

In [ ]:

In [ ]:

In [ ]:
```
students = (2,4, 5, 'oiop', (4,6,7), 'kl')
```

In [ ]:
```
#Example
my_tuple = (1, 2, 3, 4, 2, 5)
```

In [ ]:
```
my_tuple
```

In [ ]:
```
tuple2 = (1, "world", 2.71, (4, 5), {"key": "value"})
# Contains integer, string, float, tuple, and dictionary
```

In [ ]:
```
t = (1, 2, 2, 3,4,5)
```

In [ ]:
```
t.index(2)
```

In [ ]:
```
students
```

Common Tuple Methods:

- **count(item):** Returns the number of times an item appears in the tuple.
- **index(item):** Returns the index of the first occurrence of an item.

In [ ]:
```
# Count occurrences of an item

my_tuple.count(2)
```

In [ ]:
```
# Get the index of an item
my_tuple.index(3)
```

In [ ]:
```
# Access elements by index
my_tuple[0]
```

3. Set

A set is an unordered, mutable, and unindexed collection that does not allow duplicate elements. Sets are created using curly brackets {} or the set() function.

Characteristics:

- **Unordered:** There is no defined order for the elements.
- **Mutable:** You can add or remove elements from a set, but once added, elements cannot be changed.
- **No duplicates:** Sets automatically remove duplicate elements.

In [ ]:
```
names = {'Chidi', 'Patience', 'UdoChukwu', 'Salau', 'Victor', 'Sumanya', 'Fessa', '
```

In [ ]:
```
print(names)
```

In [ ]:
```
names
```

In [ ]:

In [ ]:

In [ ]:
```
# example
my_set = {1, 2,2, 3, 4, 5, 5}
```

In [ ]:
```
my_set
```

In [ ]:
```
set2 = {1, "apple", 3.14, (1, 2)}
# Contains integer, string, float, and tuple
```

In [ ]:
```
set2
```

In [ ]:
```
set2.discard(66)
```

In [ ]:
```
set2.clear()
```

Common Set Methods:

- **add(item):** Adds an element to the set.
- **remove(item):** Removes a specific element from the set.
- **discard(item):** Removes a specific element if it exists, without raising an error if it doesn't.
- **pop():** Removes and returns a random element.
- **clear():** Removes all elements from the set.
- **union(set2):** Returns a set containing all elements from both sets.
- **intersection(set2):** Returns a set containing elements found in both sets.
- **difference(set2):** Returns a set containing elements in the first set but not in the second.

In [ ]:
```
names.issubset(students)
```

In [ ]:
```
students = {3,3,3, 4,4, 'oil','man','man', 5,66,66}
```

In [ ]:
```
students.pop()
```

In [ ]:
```
st = {3,'man', 5, 7, 'kop'}
```

In [ ]:
```
st
```

In [ ]:
```
students.intersection(st)
```

In [ ]:
```
students.difference(st)
```

In [ ]:
```
# Adding an element
my_set.add(14)
```

In [ ]:
```
# Removing an element
my_set.remove(3)
```

In [ ]:
```
my_set
```

In [ ]:
```
# Set union (combines two sets)
union = my_set.union(set2)
```

In [ ]:
```
union
```

In [ ]:
```
# Set intersection (common elements)
intersection_set = my_set.intersection(set2)
```

In [ ]:
```
intersection_set
```

In [ ]:
```
# Set difference (elements in my_set not in another_set)
difference_set = my_set.difference(set2)
```

In [ ]:
```
difference_set
```

4. Dictionary

A dictionary is an ordered, mutable collection of key-value pairs. Each key must be unique, and it is associated with a specific value. Dictionaries are created using curly braces {} with key-value pairs separated by a colon :.

Characteristics:

- **Ordered:** The elements are stored as key-value pairs. Note : In the previous versions of Python, 3.6 and earlier, dictionaries are unordered.
- **Mutable:** You can add, change, or remove key-value pairs.

- **Keys must be unique:** You cannot have duplicate keys, though values can be duplicated.

In [ ]:
```
studentsinfo = {'names': ['Patience', "UdoChukwu", "Salau", 'Victor', 'Sumanya', 'F
                'Location': ['Abuja', 'Lagos', 'Enugu', 'Ibadan', 'Abuja', 'Benin',
                'Age': [45, 47, 32, 37, 26, 28, 38]}
```

In [ ]:
```
print(studentsinfo)
```

In [ ]:
```
students['Courses'] = ['excel', 'pbi','sql','py']
```

In [ ]:
```
studentsinfo
```

In [ ]:
```
my_dict = {
    "name": "Alice",
    "age": 25,
    "city": "New York"
}
```

In [ ]:
```
print(my_dict)
```

In [ ]:
```
dict1 = {"name": ["Alice", "Sam", "Obi", "Chi"],
    "age": [25, 20, 30, 45],
    "city": ["New York", "LA", "Jos", "Abj"]
}
```

In [ ]:
```
dict1
```

In [ ]:
```
dict2 = {
    "integer_key": 10,
    "string_key": "value",
    "float_key": 3.14,
    "list_key": [1, 2, 3],
    "tuple_key": (4, 5),
    "dict_key": {"inner_key": "inner_value"}
}
# Keys are string, values include integer, string, float, list, tuple, and another
```

Common Dictionary Methods:

- **get(key):** Returns the value for a specified key.
- **keys():** Returns all the keys in the dictionary.
- **values():** Returns all the values in the dictionary.
- **items():** Returns all key-value pairs as tuples.
- **update(dict2):** Updates the dictionary with the key-value pairs from another dictionary.
- **pop(key):** Removes and returns the value for the specified key.
- **clear():** Removes all key-value pairs from the dictionary.

In [ ]:
```
studentsinfo.clear()
```

In [ ]:

```

```

In [ ]:

```
studentsinfo['Gender'] = ['Female', 'Female', 'Male', 'Female', 'Female', 'Female',
```

In [ ]:

```
print(studentsinfo)
```

In [ ]:

```
studentsinfo['Score'] = [89, 90, 78, 91, 78, 89, 96]
```

In [ ]:

```
# pandas, numpy, data viz
# data frame - df
import pandas as pd
```

In [ ]:

```
d = pd.DataFrame(studentsinfo)
d
```

In [ ]:

```
pd.Series(name)
```

In [ ]:

```
dict1["age"]
```

In [ ]:

```
# Using the get() method to avoid KeyError
dict1.get("age")
```

In [ ]:

```
# Adding a new key-value pair
my_dict["profession"] = "Engineer"
```

In [ ]:

```
my_dict
```

In [ ]:

```
# Adding a new key-value pair
dict1["profession"] = ["Engineer", "Pilot", "Teacher", "House_wife"]
```

In [ ]:

```
dict1
```

In [ ]:

```
# Updating the value for an existing key
my_dict["age"] = 26
```

In [ ]:

```
my_dict
```

In [ ]:

```
# this will displace all the values in age with 20
dict1["age"] = 20
```

In [ ]:

```
dict1
```

In [ ]:

```
# to change the value of a specific value
# Step 1: Find the index of the value 30 in the 'age' list
index_to_update= dict1["age"].index(30)
```

In [ ]:

```
index_to_update
```

In [ ]:

```
# Step 2: Update the value at the found index
```

```
dict1["age"][index_to_update] = 32
```

In [ ]:
```
# Removing a key-value pair
my_dict.pop("city")
```

In [ ]:
```
my_dict
```

In [ ]:
```
# Access all keys
my_dict.keys()
```

In [ ]:
```
# Access all values
my_dict.values()
```

In [ ]:
```

```

Conditional Statements in Python

Conditional statements are used to perform **different actions based on conditions**.

`if`, `elif`, `else` Syntax:

```
if condition:
    # code to execute if condition is True
elif another_condition:
    # code to execute if the second condition is True
else:
    # code to execute if none of the conditions are True
```

In [ ]:
```
age = 8

if age >= 18:
    print("You are eligible")
else:
    print("Sorry, you're not eligible")
```

In [ ]:
```
score = 34

if score >= 90:
    print("Grade: A")
elif score >= 70:
    print("Grade: B")
elif score >= 50:
    print("Grade: C")
else:
    print("Grade: F")
```

In [ ]:
```
Age = 28
Citizen = False

if Age >= 21 and Citizen:
    print("You can vote")
```

```
else:
    print("Sorry, you can't vote")
```

In [ ]:
```
age = 17
citizen = False

if age >= 18 and citizen:
    print("You can vote")
elif age < 18 and citizen:
    print("Under age")
elif age >= 18 and not citizen:
    print("You're not a citizen")
else:
    print("You are under age and not a citizen so you can't vote")
```

In [ ]:
```
age = 42
status = "Adult" if age >= 18 else "Minor"
print(status)
```

In [ ]:

In [ ]:

In [ ]:
```
age = 45

if age < 18:
    print('You are a child')
elif age < 50:
    print('you are an adult1')
else:
    print('you are an adult2')
```

In [ ]:

In [ ]:
```
age = 20

if age >=80:
    print("You're an adult.")
```

In [ ]:
```
age = 20

if age >= 18:
    print("You're an adult.")
else:
    print("You're still a minor.")
```

In [ ]:
```
temperature = 25

if temperature > 30:
    print("It's a hot day!")
elif temperature > 20:
    print("It's a warm day!")
```

```
else:
    print("It's a cool day!")
```

In [ ]:
```
score = 75

if score >= 70:
    print("You passed with a Distinction!")
elif score >= 50:
    print("You passed.")
else:
    print("You failed.")
```

In [ ]:
```
balance = 5000

if balance > 10000:
    print("You can afford a vacation!")
elif balance >= 5000:
    print("You're doing okay.")
else:
    print("You need to save more.")
```

In [ ]:

In [ ]:
```
age = int(input("How old are you? "))

if age < 18:
    print("You're a minor.")
elif age < 65:
    print("You're an adult.")
else:
    print("You're a senior citizen.")
```

In [ ]:

The input() Function

- The input() function allows you to get user input as a string.

In [ ]:
```
Name = input("What is your name")
print(f'Hi, {Name}, nice to me you')
```

In [ ]:
```
username = input("Enter username")
password = input("Enter password")
```

In [ ]:
```
username = 'Nazom'
password = '1235'
```

In [ ]:
```
u = input("username")
p = input("password")

if u == username and p == password:
       print("login successfully")
```

```
else:
    print("incorrect details")
```

In [ ]:
```
u = input("username")
p = input("password")

if u == username and p == password:
       print("login successfully")
elif u != username:
    print("incorrect username")
elif p != password:
    print("incorrect password")
else:
    print("incorrect details")
```

In [ ]:
```

```

In [ ]:
```
name = input("Tell me your name")
print(f'Hii', {name}, 'How do you do?')
```

In [ ]:
```
username = input('username')
password = input('password')
```

In [ ]:
```
name = input("What is your name? ")
print("Hello, " + name + "!")
```

In [ ]:
```
age = int(input("Enter your age: "))
```

In [ ]:
```
Year_of_birth = int(input('What year were you born?'))
Age = 2025-Year_of_birth

print(f'You are {Age} years old')
```

In [ ]:
```
food = input("What's your favorite food? ")
print("Yum! I like " + food + " too!")
```

In [ ]:
```
Name = input('Enter username')
Pin = int(input('Enter pin'))
```

Conditional statement & Input function

In [13]:
```
yob = int(input("What's your birth year -"))
if yob > 2025:
    print ('invalid year')
elif yob < 1900:
    print("Hello Granny, check again!")
else:
    Age = 2025 - yob
    print(f"you are {Age} years old")
```

```
you are 21 years old
```

In [ ]:

In [ ]:
```
# Program for a grading system
score = int(input('Enter your score:'))

if score >= 75:
    print("You got an A - Distinction!")
elif score >= 60:
    print("You got a B - Good job!")
elif score >= 50:
    print("You got a C - Fair!")
elif score >= 40:
    print("You got a D - Needs improvement")
else:
    print("You failed - Try again.")
```

In [ ]:
```
# Program for a grading system
score = int(input('Enter your score:'))

if score > 100:
    print("Invalid score - score cannot be greater than 100.")
elif score >= 75:
    print("You got an A - Distinction!")
elif score >= 60:
    print("You got a B - Good job!")
elif score >= 50:
    print("You got a C - Fair!")
elif score >= 40:
    print("You got a D - Needs improvement")
else:
    print("You failed - Try again.")
```

In [ ]:
```
score = int(input('Enter your score:'))
if score > 100:
    print("Invalid score - score cannot be greater than 100.")
elif 70  <= score <= 100:
    print("You got an A - Distinction!")
```

In [ ]:

In [ ]:
```
Year_of_birth = int(input("What year were you born? "))
if Year_of_birth > 2025:
    print("Wrong year entry")
else:
    Age = 2025 - Year_of_birth

    print(f"You are {Age} years old")
```

In [ ]:

In [ ]:
```
Year_of_birth = int(input("What year were you born? "))
if Year_of_birth > 2025:
    print("Wrong year entry")
```

```
elif Year_of_birth < 1900:
    print("Hi time traveler, check again!")
else:
    print(f"You are {Age} years old")
```

In [ ]:
```
username = input("Enter username: ")
password = input("Enter password: ")

if username == "mee" and password == "1234":
    print("Login successful!")
else:
    print("Incorrect credentials.")
```

In [ ]:
```
# Define the correct username and pin for validation
correct_username = "mee"
correct_pin = 1234

# Get user input
Name = input('Enter username: ')
Pin = int(input('Enter pin: '))

# Check if the username is correct
if Name != correct_username:
    print("Error: Incorrect username")
# Check if the pin is correct
elif Pin != correct_pin:
    print("Error: Incorrect pin")
else:
    print("Successful login!")
```

In [ ]:

List comprehension

- is a concise and powerful way to create lists in Python.
- allows to generate new lists by applying an expression to each item in an existing iterable (like a list or range) and optionally include a condition to filter the items ie it's a concise way to create a new list by filtering or transforming elements of an existing list.
- They can also be extended to other data structures like dictionaries and sets.

In [ ]:
```
# Syntax
[expression for item in iterable if condition]
```

In [ ]:
```
range(2, 100, 2)
```

In [ ]:
```
# range(start, stop, step)
2 100
```

In [ ]:
```
e = [y for y in range(2, 101)]
print(e)
```

In [ ]:
```
[y for y in range(2, 101, 2)]
```

In [ ]:
```
 Gender = ['Male' if x%2 == 0 else "Female" for x in range(10)]
print(Gender)
```

In [ ]:
```
Gender
```

In [ ]:

In [ ]:

r = range(10) print(r)

In [ ]:
```
t = [e for e in range(100, 501, 50)]
```

In [ ]:
```
print(t*2)
```

In [ ]:
```
10/3
```

In [ ]:
```
20%2
```

In [ ]:
```
[x**2 for x in range(11)]
```

In [ ]:

In [ ]:
```
[i for i in range(10)]
```

In [ ]:
```
2%3
```

In [ ]:
```
g = ["she" if p % 2 == 0 else "he" for p in range(21)]
```

In [ ]:
```
print(g)
```

In [ ]:

In [ ]:
```
labels = ["Even" if x % 2 == 0 else "Odd" for x in range(10)]
print(labels)
```

In [ ]:
```
# Generate a list of squares of numbers from 0 to 9.
squares = [x**2 for x in range(10)]
print(squares)
```

In [ ]:
```
# Create a list of even numbers from 0 to 25.
evens = [x for x in range(25) if x % 2 == 0]
print(evens)
```

In [ ]:
```
evens_sqd = [x**2 for x in range(25) if x % 2 == 0]
print(evens_sqd)
```

In [ ]:
```
# Creating a list from an existing list
[e**2 for e in evens]
```

In [ ]:
```
# Convert a list of temperatures from Celsius to Fahrenheit.
celsius = [0, 10, 20, 30]
fahrenheit = [(t * 9/5) + 32 for t in celsius]
print(fahrenheit)
```

In [ ]:
```
# To create a list in Python containing a sequence of 0s and 1s using list comprehe
binary = [0 if i % 2 == 0 else 1 for i in range(20)]
print(binary)
```

In [ ]:
```
n = [0 if x % 2 == 0 else 1 for x in range(100)]
print(n)
```

In [ ]:
```
b = [x for x in range(2, 101, 2)]
print(b)
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:
```
# [0 for i in range(10) if i % 2 == 0 else 1]
```

In [ ]:
```
# Create the list using list comprehension
# Define the length of the list
length = 20  # Change this value to adjust the length of the list

b = [5 if i % 2 == 0 else 10 for i in range(length)]
print(b)
```

In [ ]:
```
# Dictionary Comprehension
# Create a dictionary where keys are numbers and values are their squares.
squares_dict = {x: x**2 for x in range(10)}
print(squares_dict)
```

In [ ]:
```
# Set Comprehension
# Create a set of unique characters from a list.
x = ["hi", 1, 2, 3, 3, 4, 'hi', 4, 8, 8, 9]
```

In [ ]:
```
y = {i for i in x}
print(y)
```

In [ ]:
```
# list comprehensions in Python uses a for loop
# the difference;

# Using a for loop
result = []
for i in range(5):
    result.append(i * 2)
```

```
print(result)


# Using list comprehension
result = [i * 2 for i in range(5)]
print(result)
```

LOOP

Loops are used to avoid writing repetitive code. They allow us to repeat actions based on conditions.

In [ ]:
```
# Without Loops: Manually repeating lines of code for actions that need to happen m
print("Hello, world!")
print("Hello, world!")
print("Hello, world!")
```

In [ ]:
```
# With Loops: Let the computer do the repetitive task for you, making your code mor
for i in range(2):
    print("Hello, world!")
```

for Loop

- The *for loop* is used to iterate over a sequence (such as a list, tuple, dictionary, set, or string) or other iterable objects.
- A *for loop* is used to repeat actions for a known number of items. Think of it like going through a list or checklist, doing something for each item.
- Useful when you know how many times you need to repeat an action (like a list of items to go through).

syntax - Code to execute for variable in iterable:

In [ ]:
```
range(10)
```

In [ ]:
```
r = range(2,10)
print(r)
```

In [ ]:
```
r = range(2,20,5)
print(r)
```

In [ ]:
```
for numbers in range (11):
    print(numbers)
```

In [ ]:
```
for r in range(2,10):
    print(r)
```

In [ ]:
```
for r in range (2,20,2):
    print(r)
```

In [ ]:
```
for i in range(5):
```

```
    print(i)
```

In [ ]:
```
x = []
```

In [ ]:
```
for numbers in range(2,21,2):
    x.append(numbers)
```

In [ ]:
```
x = [numbers for numbers in range(2,21,2)]
print(x)
```

In [ ]:
```
# Create an empty list to hold the values
results = []

# For loop to iterate and collect values
for i in range(5):
    results.append(i)

# Output the collected values
print(results)
```

In [ ]:
```
x = [i for i in range (10, 50)]
p = [i for i in x if i % 2 == 0]
y = [i for i in x if i % 2 != 0]
print (x)
print (p)
print (y)
```

In [ ]:
```
for i in range(50):
    if i % 2 != 0:
        print (i)
```

In [ ]:
```
y = []
```

In [ ]:
```
for i in range(50):
    if i % 2 != 0:
        y.append(i)
print(y)
```

In [ ]:
```
y = [i for i in range(50) if i % 2 != 0]
```

In [ ]:
```
y
```

In [ ]:
```
for i in range(0, 101, 2):
    print(i)
```

In [ ]:
```
for i in range(10,0,-1):
    print(i)
print("Happy New Year!")
print("Its 2025!!!! Ahhhhh")
print("yreeeee")
```

In [ ]:
```
# sum of all even numbers from 1 to 21
total = 0
for i in range(1, 1000):
    if i % 2 == 0:
        total += i
print(f"Sum of even numbers: {total}")
```

In [ ]:
```
x = 5
x += 5
print(x)
```

In [ ]:
```
x = x+5
```

In [ ]:
```
j = [i for i in range (0,40, 2)]
j
```

In [ ]:
```
x = 5
x= x+6
print(x)
```

In [ ]:
```
x = 5
x += 6 # This is equivalent to x = x + 6
print(x)
```

In [ ]:
```
f = [i for i in range (2, 30, 2)]
```

In [ ]:
```
f
```

In [ ]:
```
d = []
for i in f:
    if i <= 20 :
        d.append(i)
d
```

In [ ]:
```
# The positioning of print(), increment, and break in the loop controls which value
```

In [ ]:
```
for i in f:
    if i == 20:
        continue
    print (i)
```

In [ ]:
```
for x in f:
    if x % 2 == 0:
        x+=2
        print(x)
```

In [ ]:
```
for x in "banana":
  print(x)
```

In [ ]:
```
names = ['Alice', 'Bob', 'Charlie', 'jhh']
for name in names:
```

```
    print(f"Hello, {name}!")
```

In [ ]:
```
# this creates a new list with fruits containing letter 'a'.
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
afruits = []
for x in fruits:
  if "a" in x:
    afruits.append(x)

print(afruits)
```

In [ ]:
```
# using list comprehension for same
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
newlist = [x for x in fruits if "a" in x]
print(newlist)
```

In [ ]:
```
fruits = ['apple', 'banana', 'cherry']
for fruit in fruits:
    print(fruit)
```

In [ ]:
```
fruits = ["apple", "Guava", "Avacado", "banana", "cherry", "pear", "orange", "banan
for x in fruits:
  print(x)
  if x == "banana":
    break
```

In [ ]:
```
fruits = ["apple", "Guava", "Avacado", "banana", "cherry", "pear", "orange", "banan
for x in fruits:
  if x == "banana":
    break
```

In [ ]:
```
fruits = ["apple", "Guava", "Avacado", "banana", "cherry", "pear", "orange", "banan
for x in fruits:
  if x == "banana":
    continue
  print(x)
```

In [ ]:
```
countries = ["NIGERIA", "GHANA", "TOGO", "ALGERIA", "MOROCCO", "MALAWI", "BOTSWANA"
```

In [ ]:
```
countries2 = []
for x in countries:
    if "A" in x:
        countries2.append(x)
print(countries2)
```

In [ ]:
```
d = [x for x in countries if "O" in x]
print(d)
```

In [ ]:
```
countries = ["NIGERIA", "GHANA", "TOGO", "ALGERIA", "MOROCCO", "MALAWI", "BOTSWANA"
o_countries = [country for country in countries if "O" in country]
a_countries = [country for country in countries if "A" in country]
print(o_countries)
print(a_countries)
```

In [ ]:
```
person = {'name': 'Alice', 'age': 25, 'city': 'New York'}
for key, value in person.items():
    print(f"{key}: {value}")
```

In [ ]:
```
customers = ['Alice', 'Bob', 'Charlie', 'David']

for customer in customers:
    print(f"Sending email to {customer}...")
```

In [ ]:
```
g = [4,6,8,9,]
print(sum(g))
```

In [ ]:
```
grades = [85, 90, 78, 92, 88]
print(sum(grades)/len(grades))
```

In [ ]:
```
# List of student grades
grades = [85, 90, 78, 92, 88, 65, 55, 72, 67, 80]

# Filter grades to include only those 70 and above
A70 = [grade for grade in grades if grade >= 70]
print(A70)
```

In [ ]:
```
# Calculate the average grade of the filtered list
if A70:  # Check if the filtered list is not empty
    total = sum(A70)
    average_grade = total / len(A70)
    print(f"The average grade for scores 70 and above is: {average_grade:.2f}")
else:
    print("No grades are 70 or above.")
```

In [ ]:
```
# Dictionary of employees and their salaries
employees = {
    "Alice": 70000,
    "Bob": 65000,
    "Charlie": 60000
}

# Apply a 10% raise
for name, salary in employees.items():
    new_salary = salary * 1.10
    employees[name] = new_salary

print("Updated salaries:")
for name, salary in employees.items():
    print(f"{name}: ${salary:.2f}")
```

In [ ]:
```
(10/100 * 70000)+ 70000
```

In [ ]:
```
# List of invoices, each with item and amount
invoices = [
    {"item": "Laptop", "amount": 1200},
    {"item": "Mouse", "amount": 25},
    {"item": "Keyboard", "amount": 75}
]

# Calculate the total amount
total_amount = 0
for invoice in invoices:
    total_amount += invoice["amount"]

print(f"Total amount of all invoices: ${total_amount:.2f}")
```

In [ ]:
```
# Tuple of daily temperatures for a week
temperatures = (72, 75, 79, 80, 76, 73, 71)

# Find the highest temperature
highest_temp = temperatures[0]
for temp in temperatures:
    if temp > highest_temp:
        highest_temp = temp

print(f"The highest temperature of the week is: {highest_temp}°F")
```

In [ ]:

In [ ]:

While Loop

- The while loop repeatedly executes a block of code as long as a condition is True ie repeats an action as long as a condition remains true. - It's like setting a task that repeats until something specific happens.# syntax - Code to execute while condition:

In [ ]:

In [ ]:
```
# Print i as long as i is less than 6:
i = 1
while i <= 100:
  print(i)
  i += 1
```

In [ ]:
```
i = 1
while i < 10:
  print("Hello World")
  i += 1
```

In [ ]:
```
#you can stop the loop even if the while condition is true with the break statement
# Example: Exit the loop when i is 3:

i = 1
while i < 6:
```

```
  print(i)
  if i == 3:
    break
  i += 1
    # The print(i) comes before the `if` check for `break`. Therefore, the number 3
```

In [ ]:
```
# Using while with a Break Statement:
i = 0
while True:
    print(i)
    i += 1
    if i == 3:
        break
```

In [ ]:
```
# The continue statement allows you to stop the current iteration, and continue wit
# Example: Continue to the next iteration if i is 3:

i = 0
while i < 6:
  i += 1
  if i == 3:
    continue
  print(i)
```

In [ ]:
```
i = 2
while i < 21:
    if i % 2 == 0:
        print(i)
    i =  i + 1
```

In [ ]:
```
e = []
i = 2
while i < 21:
    if i % 2 == 0:
        e.append(i)
    i = i + 1

print(e)
```

In [ ]:
```
#Print a message once the condition is false:
i = 1
while i < 6:
  print(i)
  i += 1
else:
  print("i is no longer less than 6")
```

In [ ]:
```
countdown = 10

while countdown >= 0:
    print(f"Almost there: {countdown}")
    countdown -= 1

print("Time's up!")
```

In [ ]:
```
# Prints all letters except 'e' and 's'
i = 0
a = 'HelloWorld'

while i < len(a):
    if a[i] == 'l' or a[i] == 'W':
        i += 1
        continue

    print('Current Letter :', a[i])
    i += 1
```

In [ ]:
```
secret_number = 7
guess = None

while guess != secret_number:
    guess = int(input("Guess the number: "))
    if guess < secret_number:
        print("Too low!")
    elif guess > secret_number:
        print("Too high!")
print("You got it!")
```

In [ ]:
```
shopping_list = ["apples", "bananas", "oranges", "milk", "bread"]

while shopping_list:
    item = shopping_list.pop(0)  # Remove the first item from the list
    print(f"Processing item: {item}")

print("All items have been processed!")
```

In [ ]:
```
shopping_list
```

In [ ]:
```
correct_password = "code045"
password = ""

while password != correct_password:
    password = input("Enter the password: ")
    if password == correct_password:
        print("Access granted!")
    else:
        print("Wrong password. Try again.")
```

In [ ]:
```
# Task: Asking a User for Correct Login Credentials
# A login system that keeps asking for credentials until the correct ones are enter
correct_password = "c35k"
attempts = 3

while attempts > 0:
    password = input("Enter your password: ")
    if password == correct_password:
        print("Login successful!")
        break
    else:
```

```
        attempts -= 1
        print(f"Wrong password! {attempts} attempts left.")

if attempts == 0:
    print("Account locked due to too many failed attempts.")
```

In [ ]:

```

```

In [ ]:

```
# atm example
balance = 500
withdraw = True

while balance > 0 and withdraw:
    print(f"Your current balance is: ${balance}")
    amount = float(input("Enter amount to withdraw (or 0 to stop): "))

    if amount == 0:
        print("Exiting withdrawal process.")
        withdraw = False
    elif amount > balance:
        print("Insufficient funds. Try a smaller amount.")
    else:
        balance -= amount
        print(f"You have successfully withdrawn ${amount}. Remaining balance: ${bal

if balance == 0:
    print("Your account balance is now zero. No further withdrawals allowed.")
```

In [ ]:

```
import random

correct_number = random.randint(1, 10)  # Generate a random number between 1 and 10
guess = None

while guess != correct_number:
    guess = int(input("Guess the number (between 1 and 10): "))
    if guess < correct_number:
        print("Too low! Try again.")
    elif guess > correct_number:
        print("Too high! Try again.")

print(f"Congratulations! You guessed the correct number: {correct_number}")
```

In [ ]:

```
# for vs while
for i in range(1, 6):
    print(i)
```

In [ ]:

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

In [ ]:

```
# Example: Checking inventory for multiple items until a condition is met
inventory = {"apple": 5, "banana": 8, "cherry": 0}
items_to_check = ["apple", "banana", "cherry"]
```

```
all_in_stock = False

while not all_in_stock:
    all_in_stock = True
    for item in items_to_check:
        if inventory[item] == 0:
            print(f"{item} is out of stock!")
            all_in_stock = False
            break

    if all_in_stock:
        print("All items are in stock.")
        break
    else:

        for item in items_to_check:
            if inventory[item] == 0:
                inventory[item] += 5

print("Final inventory:", inventory)
```

In [ ]:

def

- def is a keyword used to define a function.
- A function is a block of reusable code that performs a specific task.
- Functions allow you to group a set of statements together that can be executed whenever you call the function.
- Functions help in organizing code, making it modular, readable, and easier to manage.

In [ ]:
```
calculate_revenue(567788, 56)
```

In [ ]:
```
def calculate_revenue (price, quantity):
    return price * quantity
```

In [ ]:
```
def greet(name):
    return f"Hello, Good morning {name}!"
```

In [ ]:
```
greet('Ada')
```

In [ ]:
```
greet("Chi Oko")
```

In [ ]:
```
def multiply(e, t):
    return e*t
```

In [ ]:
```
def add(*args):
    return sum(args)
```

In [ ]:

In [ ]:

```

```

In [ ]:

```
def Greet(name, surname):
    print(f" Hi! {name} {surname}, how are you doing")
```

In [ ]:

```
Greet("Majesty", "Yolo")
```

In [ ]:

```
def greet():
    print("Hello, welcome to Python programming!")
```

In [ ]:

```
greet()
```

In [ ]:

```
# function with parameter
def greet_user(name):
    print(f"Hello, {name}! Welcome to Python programming!")
```

In [ ]:

```
# Call the function with a specific name
greet_user("Dee")
```

In [ ]:

```
# function with parameters
def g(name, surname):
    print(f"Hello! {name} {surname}! Welcome to our class.")
```

In [ ]:

```
g("Jane", "Ok")
```

In [ ]:

```
# Functions with Return Values
def add(a, b):
    return a + b
```

In [ ]:

```
add(10, 6)
```

In [ ]:

```
a = add(100777, 7868)
print(a)
```

In [ ]:

```
a % 2
```

In [ ]:

```
# Call the function and store the result
r = add(55, 30)
print(r)
```

In [ ]:

```
r*6
```

In [ ]:

```
def m5(x):
  return 5 * x
```

In [ ]:

```
m5(45)
```

In [ ]:

```
def square(x):
    return x ** 2
```

In [ ]:
```
square(455)
```

In [ ]:
```
def modulus(x):
    return x % 9
```

In [ ]:
```
modulus(45)
```

In [ ]:
```
def ad():
    x = 2545678855
    y = 1045556
    result  =  x + y
    return result
```

In [ ]:
```
ad()/88
```

In [ ]:
```
# with default parameter
def multiply(a, b=2):
    return a * b
```

In [ ]:
```
multiply(5)
```

In [ ]:
```
def divide(a, b):
    if b == 0:
        return "Cannot divide by zero!"
    return a / b

result = divide(10555, 7)
print(f"{result:.1f}")
```

In [ ]:
```
divide(9,9)
```

In [ ]:
```
def add(*a):
    return sum(a)
```

In [ ]:
```
add(2333444444, 566789985, 56788,33457,67888)
```

In [ ]:
```
add(5,7,9,34)
```

In [ ]:
```
def add_numbers(*args):
    total = 0
    for num in args:
        total += num
    return total
```

In [ ]:
```
add_numbers(5, 10, 15, 20)
```

In [ ]:
```
def sum(x,y):
    return(y-x)
```

In [ ]:
```
sum(4,5)
```

In [ ]:
```python
def concatenate(*args):
    return " ".join(args)
```

In [ ]:
```python
concatenate("Welcome", "to", "Skilharvest!")
```

In [ ]:
```python
def d(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

d(name="Sam", age=32, city="Lagos")
```

In [ ]:
```python
def database():
    print("Hello welcome to SkilHarvest, Kindly create an account")
    account = dict()
    username = str(input("Enter your prefared username "))
    password = str(input("Enter your choice password"))
    account[username] =  password
    return account
```

In [ ]:

In [ ]:
```python
# Step 1: Get user inputs
first_name = input("Enter your first name: ")
surname = input("Enter your surname: ")
age = int(input("Enter your age: "))
height = float(input("Enter your height in meters: "))
experience = input("Do you have programming experience? (yes/no): ").lower()

# Step 2: Validation
if age <= 0:
    print("Age cannot be zero or negative!")
if height < 0.5 or height > 2.5:
    print("Please enter a realistic height between 0.5m and 2.5m.")

# Step 3: Display the user's information
print("\nSummary:")
print(f"Hello, {first_name} {surname}!")
print(f"You are {age} years old, and your height is {height} meters.")
print(f"Programming experience: {'Yes' if experience == 'yes' else 'No'}")
```

In [ ]:
```python
weight = float(input("Enter your weight in kg: "))
height = float(input("Enter your height in meters: "))

bmi = weight / (height ** 2)

print(f"Your BMI is: {bmi:.2f}")

if bmi < 18.5:
    print("You are underweight.")
elif bmi <= 25.0:
    print("You have a normal weight.")
elif bmi <= 29.9:
    print("You are overweight.")
```

```
else:
    print("You are in the obesity category.")
```

In [ ]:
```
password = input("Enter a password: ")

if len(password) >= 8 and any(char.isdigit() for char in password) and any(char.isa
    print("Password is valid.")
else:
    print("Password is invalid. It must be at least 8 characters long and contain b
```

In [ ]:
```
age = int(input("Enter your age: "))
days_lived = age * 365

print(f"You have lived approximately {days_lived} days.")
```

In [ ]:
```
celsius = float(input("Enter the temperature in Celsius: "))
fahrenheit = (9/5) * celsius + 32

print(f"The temperature in Fahrenheit is {fahrenheit:.2f}")
```

Numpy (Numerical Python)

- It's a Python library used for working with arrays. An array is like a list, but it can store large amounts of data much more efficiently.
- Python lists can be slow and take up more memory when working with large data but NumPy helps solve this problem by:
  - Storing data efficiently.
  - Performing calculations faster.
  - Offering built-in functions for common mathematical and statistical operations.
- Arrays are important in data science because they allow you to store data in two-dimensional or multi-dimensional forms, similar to rows and columns in a spreadsheet.
- NumPy arrays can be used to perform operations on entire arrays at once, instead of looping through elements one by one.

In [ ]:
```
# pip install numpy
```

In [ ]:
```
import numpy as np
```

In [ ]:
```
p = [i for i in range (100, 150) if i % 2 == 0]
q = [i for i in range (2, 50) if i % 2 != 0]

print(p)
print(q)
```

In [ ]:
```
# suming two list
a = p + q

print(a)
```

In [ ]:
```
# e * o
```

In [ ]:
```
r = []

for i in range(len(e)):
     r.append(e[i] * o[i])

print(r)
```

In [ ]:
```
 # using list comprehension, elements of the two lists can be paired using the zip(

r = [x * y for x, y in zip(e, o)]
print(r)
```

In [ ]:
```
len(r)
```

In [ ]:
```
np.
```

Creating array

In [ ]:
```
 v = np.array([23, 444, 555, 666, 77,87, 9,8, 444])
m = np.array([555, 666,32,44,5,9,77,44,56])

print(np.mean(v))
```

In [ ]:
```
np.array([9,0,7,8,6])
```

In [ ]:
```
np.array([5, 4, 66, 7, 8, 2,1,5,8,90])
```

In [ ]:
```
b = np.array(e)
d = np.array(o)

print(b)
print(d)
```

In [ ]:
```
f = np.array(p)
print(f)
```

Array Operations

In [ ]:
```
np.sum([b])
```

In [ ]:
```
np.mean([d])
```

In [ ]:
```
c = b+d

print (c)
```

In [ ]:
```
y = b /d
print(y)
```

In [ ]:
```
# to summarize the decimals
np.around(y, decimals=3)
```

In [ ]:
```
y.shape
```

Array dimensions

Axes: Directions along which the data is arranged.

- 1D array has 1 axis.
- 2D array has 2 axes: rows and columns.
- 3D array has 3 axes: layers, rows, and columns.

In [ ]:
```
np.array([[b, d]])
```

In [ ]:
```
n = np.array([[b], [d],[c]])
print(n)
```

In [ ]:
```
print(n.shape)
print(n.ndim)
print(n.size)
```

In [ ]:
```
j = np.array([[7,8,9,8,0], [8,6,5,78,9]])
print(j.shape)
print(j)
```

In [ ]:
```
j.ndim
```

In [ ]:
```
# To create a 2D array with 20 elements (4 rows, 5 columns)
l = np.arange(20).reshape(4, 5)
print(l.shape)
print(l)
```

In [ ]:
```
np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])
```

In [ ]:
```
np.arange(1,37).reshape(6,6)
```

In [ ]:
```
q = np.arange(12).reshape(2, 3, 2)
print(q)
```

In [ ]:
```
np.transpose(np.arange(1,37).reshape(6,6))
```

In [ ]:
```
# indexing
l[3]
```

In [ ]:
```
# accessing an element using its row and column index
l[1,3]
```

In [ ]:
```
l[2]*l[0]
```

In [ ]:
```
# adding index of two different arrays
l[3] + y[3]
```

In [ ]:
```
q[0]
q[0,2]
q[1,2,1]
```

In [ ]:
```
q[0,2]
```

List of data types in NumPy and the characters used to represent them.

- S - string
- i - integer
- b - boolean
- f - float
- c - complex float
- m - timedelta
- M - datetime
- O - object
- u - unsigned integer
- U - unicode string
- V - fixed chunk of memory for other type (void)
  - size can be defined for i, u, f, S and U

In [ ]:
```
# to get the data type of an array
b.dtype
```

In [ ]:
```
# specifying the data type of an array
v = np.array([1, 2, 3, 4], dtype='S')
print(v)
```

In [ ]:
```
v.dtype
```

In [ ]:
```
# data type conversion
v.astype(int)
```

Some of the most important NumPy methods

1. **Creating Arrays**
   - a. np.array(): to create a NumPy array from a Python list or tuple.
   - b. np.zeros(), np.ones(), np.full(): create arrays filled with zeros, ones, or a specific value.
   - c. np.arange(), np.linspace(): create arrays of evenly spaced values.

In [ ]:
```
r= np.array([[3, 6, 8, 9, 7, 5], [7,2,8.0,9.7, 4, 12], [4,8,7,9,0,6], [7, 8, 9, 0,
print(r)
```

In [ ]:
```
np.zeros(((4, 2, 2)))
```

In [ ]:
```
np.zeros((2, 15))
```

In [ ]:
```
np.ones((10))
```

In [ ]:
```
np.full((3, 4, 4), 1)
```

In [ ]:
```
np.arange(0, 100, 10 ) # [start, stop, step]
```

In [ ]:
```
np.linspace(2, 10, 10)# [start, stop, num_points]
```

In [ ]:
```
np.linspace(2, 10, 10).round(2)
```

In [ ]:
```
t = np.linspace(2, 10, 10)
print(t)
```

In [ ]:
```
t = np.round(t, decimals = 3)
print(t)
```

2. Reshaping and Manipulating Arrays

- np.reshape(): reshapes an array without changing its data
- np.ravel(), np.flatten(): converts multi-dim. arrays into 1D arrays. ravel() returns a view, and flatten() returns a copy
- np.transpose(): reverses or permutes the axes of an array, often used with matrices.

In [ ]:
```
print(b)
```

In [ ]:
```
v = b.reshape(2, 6, 2)
```

In [ ]:
```
p = np.arange(0, 100).reshape(10,10)
```

In [ ]:
```
p
```

In [ ]:
```
p.ravel()
```

In [ ]:
```
j.reshape(5,2)
```

In [ ]:
```
np.ravel(j)
```

In [ ]:
```
l.ravel()
```

In [ ]:
```
j.flatten()
```

In [ ]:
```
np.transpose(l)
```

In [ ]:
```
p
```

In [ ]:
```
p.transpose()
```

In [ ]:
```
l
```

3. Basic Math and Statistics Operations

- np.sum(), np.mean(), np.median(): compute the sum, mean, and median of an array.
- np.max(), np.min(), np.argmax(), np.argmin(): find the maximum, minimum values and their indices in an array.
- np.cumsum(), np.cumprod(): return the cumulative sum or product along an array.

In [ ]:
```
t = np.sum(p)
```

In [ ]:
```
t
```

In [ ]:
```
p
```

In [ ]:
```
np.median(p)
```

In [ ]:
```
p.std()
```

In [ ]:
```
np.sum(j)
```

In [ ]:
```
p.sum()
```

In [ ]:
```
# summing two arrays
b
```

In [ ]:
```
d
```

In [ ]:
```
b+d
```

In [ ]:
```
# adding a number to every element in an array - vectorization
b+10
```

In [ ]:
```
b[5]+b[8]
```

In [ ]:
```
#indexing
b[2]+p[0,2]
```

In [ ]:
```
b
```

In [ ]:
```
p
```

In [ ]:
```
b[4]+b[23]
```

In [ ]:
```
p.max()
```

In [ ]:
```
v.argmax()
```

In [ ]:
```
p
```

In [ ]:
```
b
```

In [ ]:
```
b.cumprod()
```

In [ ]:
```
type(b)
```

In [ ]:
```
v.cumprod()
```

In [ ]:
```
V = np.array(v, dtype=np.float64)
```

In [ ]:
```
 V.cumprod()
```

4.Sorting and Logical Operations

- np.sort()
- np.where(): Returns the indices of elements that satisfy a condition.
- np.any() and np.all(): Tests whether any or all elements of an array evaluate to True.

In [ ]:
```
r
```

In [ ]:
```
 np.sort(r)
```

In [ ]:
```
j
```

In [ ]:
```
# Find the index of the element
np.where(j == 8)
```

In [ ]:
```
b
```

In [ ]:
```
np.where(j > 6)
```

In [ ]:
```
b[b>6]
```

First array (array([0, 0, 0, 0, 1, 1, 1], dtype=int64)): are the row indices where the condition j > 6 is true. Second array (array([0, 1, 2, 3, 0, 3, 4], dtype=int64)): are the column indices where the condition is true. j[0, 0] = 7 j[0, 1] = 8 j[0, 2] = 9 j[0, 3] = 8 j[1, 0] = 8 j[1, 3] = 78 j[1, 4] = 9

In [ ]:
```
j[j > 6]
```

In [ ]:
```
p[p<=5]
```

In [ ]:
```
p[p>=50]
```

In [ ]:
```
b[b==30]
```

In [ ]:
```
b
```

In [ ]:
```
d[d!=31]
```

In [ ]:
```
[b[b>30], d[d>31]]
```

In [ ]:
```
np.array([b[b>30], d[d>31]])
```

In [ ]:
```
# what 2 arrays have in common
np.intersect1d(b, v)
```

In [ ]:
```
np.any(j<50)
```

In [ ]:
```
np.all(j<50)
```

5. Removing Elements from an Array

- np.delete()

In [ ]:
```
b
```

In [ ]:
```
# Removing Elements from a 1D Array
np.delete(b,[2, 7,8])
```

In [ ]:
```
np.delete(b, np.arange(2, 10))

# np.delete(b, [2:5])
```

In [ ]:
```
p
```

In [ ]:
```
# Removing rows and columns from a 2D Array
np.delete(p, 0, axis=0)  # axis=0 for rows
```

In [ ]:
```
np.delete(p, 1, axis=1)  # axis=1 for columns
```

In [ ]:
```
v.shape
```

In [ ]:
```
v
```

In [ ]:
```
b.ndim
```

In [ ]:
```
np.delete(v, 1, axis=0)
```

In [ ]:
```
v
```

In [ ]:
```
# for 3D
np.delete(V, 0, axis=2)  # axis=0 for layers, axis =1 for rows, axis = 2 for column
```

In [ ]:
```
V
```

6. Joining and appending array(s)

- np.concatenate() used to join two or more arrays along an existing axis, doesn't add new dimensions—just extends the array along the specified axis
- np.stack() is used to join arrays along a new axis. This is different from concatenation because it creates a new dimension in the resulting array
- np.append()

In [ ]:
```
np.append(b, 7)
```

In [ ]:
```
 e = np.concatenate ((b,d), axis =0)
print(e)
```

In [ ]:
```
np.sort(e)
```

In [ ]:
```
q = np.arange(1, 21).reshape(10,2)
```

In [ ]:
```
q
```

In [ ]:
```
p
```

In [ ]:
```
np.concatenate ((p,q), axis =1)
```

In [ ]:
```
p
```

In [ ]:
```
np.concatenate ((b,d), axis = 0)
```

In [ ]:
```
 np.stack((b,d), axis =0)
```

In [ ]:
```
# np.stack((b, [3,4,5,6,7,7]),axis = 0)
```

In [ ]:
```
# np.vstack() stacks arrays vertically - first axis (rows). Works like concatenatio
np.vstack((b,d))
```

In [ ]:
```
#np.hstack() stacks arrays horizontally - second axis (columns). Works like concate
np.hstack((b,d))
```

In [ ]:
```
np.append(b, [3,4, 5, 6])
```

In [ ]:
```
np.append(p, np.arange(0,10))
```

7 splitting array(s)

- np.split() splits an array into multiple sub-arrays equally along a specified axis.
- np.array_split() is a more flexible version of np.split() because it allows for unequal splits.

In [ ]:
```
np.split(b, 4) # Split into 3 equal parts
```

In [ ]:
```
# For 2D array - split into 2 parts along axis=1 (columns), axis = 0 (rows)
np.split(p, 2, axis=1)
```

In [ ]:
```
np.hsplit(p, 2)
```

In [ ]:
```
np.split(p, 2, axis=0)
```

In [ ]:
```
np.vsplit(p, 2)
```

In [ ]:
```
np.split(p, 2, axis = 1)
```

In [ ]:
```
np.array_split(p, 3, axis = 1)
```

In [ ]:
```
v
```

In [ ]:
```
# For 3D arrays (or higher), you can split them depth-wise using np.dsplit(), which
np.dsplit(v, 2)
```

8. Random Number Generation

- np.random.rand() and np.random.randint(): generates random numbers (float and integers).

In [ ]:
```
np.random.rand(20)  # 4 random floats
```

In [ ]:
```
np.random.rand(4).round(2)
```

In [ ]:
```
np.round(np.random.rand(4), decimals =2)
```

In [ ]:
```
np.random.randint(10, 100, 55)  # 5 random integers between 50 and 100
```

9. Linear Algebra Functions/Matrix Operations: These are essential in data science for tasks like regression analysis, machine learning, and simulations.

- np.linalg.inv() (matrix inverse)
- np.linalg.det() (matrix determinant)
- np.dot() or np.matmul() for matrix multiplication
- np.linalg.eig() for eigenvalues and eigenvectors

In [ ]:
```
p.shape
```

In [ ]:
```
j.shape
```

In [ ]:
```
np.dot(v,j)
```

In [ ]:
```
# matrix multiplication - doesn't support 1D arrays.
np.matmul(v,j)
```

In [ ]:
```
np.matmul(b,d)
```

In [ ]:
```
b
```

In [ ]:
```
d
```

In [ ]:
```
np.dot(b,d)
```

In [ ]:
```
np.dot(v,j)
```

In [ ]:
```

```

Pandas

- Pandas is a Python library used for data manipulation and analysis.
- A pandas Series is a one-D labeled array that can hold any data type. It is like a column in a DataFrame and each of its element is associated with an index.
- DataFrame is a 2-dimensional labeled data structure with columns of potentially different types and rows.

In [ ]:
```
import pandas as pd
```

In [ ]:
```
students
```

In [310…
```
s = pd.DataFrame(studentsinfo)
```

In [ ]:
```
print(studentsinfo)
```

In [ ]:
```
# Creating a Series from a List
p = [i for i in range(1, 11)]
```

In [287…
```
s.head(3) # first 5 rows
```

Out[287…

	names	Location	Age	Gender
0	Patience	Abuja	45	Female
1	UdoChukwu	Lagos	47	Female
2	Salau	Enugu	32	Male

In [ ]:
```
s
```

In [ ]:
```
# s.shape - # number of rows and columns
# s.columns -
# s.info()
# s.describe()  # stats of numerical columns
s.describe(include = 'all')
```

In [309…

Out[309…

	names	Location	Age	Gender
0	Patience	Abuja	32	Female
1	UdoChukwu	Lagos	32	Female
2	Salau	Enugu	32	Male
3	Victor	Ibadan	32	Female
4	Sumanya	Abuja	32	Female
5	Fessa	Benin	32	Female
6	Jennifer	Asaba	32	Female

In [293…
```
s.names
```

Out[293…
```
0    45
1    47
2    32
3    37
4    26
5    28
6    38
Name: Age, dtype: int64
```

In [ ]:
```
pd.Series([[3,4,5,6], [8,3,4,5]])
```

In [ ]:
```
s[8]
```

In [ ]:
```
s = pd.Series(p, index = ['a', 'b','c','d','e','f','g','h','i','j'])
```

In [ ]:
```
s['e']-s['c']
```

In [ ]:
```
s[5]-s[4]
```

In [ ]:
```
# Creating a Series with Custom Index
s = pd.Series(p, index=['a', 'b','c','d','e','f','g','h','i','j'])
print(s)
```

In [ ]:
```
s['i']
```

In [ ]:
```
s.describe()
```

In [ ]:
```
dsc = {"Name": ['Kapo Kapo', "Abiodun Tumi", " Bless Ogo", "Yahi Yahi", " Majesty Y
       "Gender": ["F", "F", "F", "M", "M"],
       "Age": [ 32, 34, 25, 45, 16],
       "Location": ["Abj", "Lag", "PH", "Ben", "US"],
       "Religion": ["Chr", "Mus", "Mus", "Chr", "Chr"]}
```

In [ ]:
```
dsc
```

In [ ]:
```
df = pd.DataFrame(dsc)
```

In [ ]:
```
df
```

In [ ]:
```
df.Name
```

In [ ]:
```
df["Name"]
```

In [ ]:
```
len(df)
```

In [ ]:
```
df.count()
```

In [ ]:
```
df.head()
```

In [ ]:
```
df.tail()
```

In [ ]:
```
df.sample()
```

In [ ]:
```
df.columns
```

In [ ]:
```
D
```

In [ ]:
```
df.shape #Returns the number of rows and columns in the DataFrame.
```

In [ ]:
```
df.info()  # Summary of the data
```

In [ ]:
```
df.describe()
```

In [ ]:
```
df.describe().T # T stands for transpose.
```

In [ ]:
```
# add new column
df['Group'] = [1, 2, 3, 4, 5]
```

In [ ]:
```
df
```

In [ ]:
```
# To add new rows;
#first create a dict
n = pd.DataFrame({'Name': ['David Yul', 'Angie Okon'],
                        'Gender': ['Male', 'Female'],
                        'Age': [40, 19],
                        'Location': ['Seattle', 'Rives'],
```

```
                                'Group':[4, 3]})
print(n)
```

In [ ]:
```
#then join it to the dataframe
df = pd.concat([df, n], ignore_index=True)
```

In [ ]:
```
df
```

In [ ]:
```
#to remove column(s)
df.drop('Age', axis=1)
```

In [ ]:
```
df.drop(['Age', 'Location'], axis=1)
```

In [ ]:
```
df.drop(columns=['Age', 'Group'])
```

In [ ]:
```
df.drop(df.index[:3])  # Remove the first 3 rows
```

In [ ]:
```
df.drop([1, 3])
```

In [ ]:
```
df['Age'].mean()
```

In [ ]:
```
df["Gender"] = df["Gender"].replace('f','Female')
```

In [ ]:
```
df
```

In [ ]:
```
df.Gender = df.Gender.replace("m", "Male").replace('f', 'Female')
```

In [ ]:
```
df = df.rename(columns = {"Name": "Full_Name"})
```

In [ ]:
```
# df.rename(columns={'Name': 'Full name'}, inplace = True)
```

In [ ]:
```
df['Age'].astype(int)
```

Indexing - selecting rows and columns from a DataFrame or Series using iloc[] and loc[]

- **iloc[]** - for positional indexing, selects data based on integer-based positions. Exclusive of the end index.
- **loc[] (Label Location)** - for label-based indexing. allows selecting rows and columns based on labels. Inclusive of the end index.

In [311…
```
s
# df.iloc[0]
```

Out[311…

	names	Location	Age	Gender
0	Patience	Abuja	45	Female
1	UdoChukwu	Lagos	47	Female
2	Salau	Enugu	32	Male
3	Victor	Ibadan	37	Female
4	Sumanya	Abuja	26	Female
5	Fessa	Benin	28	Female
6	Jennifer	Asaba	38	Female

In [ ]:
```
df.iloc[:, 0]
```

In [ ]:
```
df.iloc[0:2, 0:2]
```

In [ ]:
```
df.iloc[3:5, 2:4]
```

In [ ]:
```
df.iloc[:4, 2:]
```

In [ ]:
```
df.iloc[0:4]
```

In [ ]:
```
print(df.iloc[0, 2]) # Access a specific value: row index 0, Column index 2
```

In [ ]:
```
df.loc[0]
```

In [ ]:
```
df.iloc[1:3]
```

In [ ]:
```
df.loc[1:3]
```

In [ ]:
```
df.loc[1:3, ["Gender", "Group"]]
```

In [ ]:
```
df.loc[0:1, ['Name', 'Age']]
```

In [ ]:
```
df.loc[D['Age'] > 25]  # Conditional filtering
```

In [ ]:
```
df.loc[D["Full_Name"] == 'Kapo Kapo']
```

In [ ]:
```
df.loc[df["Location"] == 'Abroad']
```

In [ ]:
```
df[(df['Age'] > 25) & (df['Group'] > 2)]
```

In [ ]:
```
df[(df['Age'] > 25) | (df['Group'] > 2)]
```

In [ ]:
```
df[(df['Age'] > 25) | (df['Group'] > 2)]
```

In [ ]:
```
df.loc[2:4]
#df.loc[2:4, 0:3]
```

In [ ]:
```
# Importing data
import pandas as pd
sp = pd.read_csv(r"C:\Users\ADMIN\Desktop\Dataset\Supermart Grocery Sales - Retail
```

In [ ]:
```
sp
```

In [ ]:
```
# use to_string() to print the entire DataFrame.
print(sp.to_string())
```

In [ ]:
```
# Check  your system's maximum returned rows
print(pd.options.display.max_rows)
```

In [ ]:
```
# To Increase the maximum number of rows to display the entire DataFrame:
pd.options.display.max_rows = 60
```

In [ ]:
```
sp
```

1. Basic Information about the DataFrame

In [ ]:
```
len(sp)
```

In [ ]:
```
sp.count()
```

In [ ]:
```
sp.head(20)
```

In [ ]:
```
sp.tail(8)
```

In [ ]:
```
sp.sample(6)
```

In [ ]:
```
sp.columns
```

In [ ]:
```
sp.shape #Returns the number of rows and columns in the DataFrame.
```

In [ ]:
```
sp.info()  # Summary of the data
```

In [ ]:
```
sp.describe()
```

In [ ]:
```
sp.describe().T # T stands for transpose.
```

In [ ]:
```
sp["Order Date"].info()
```

In [ ]:
```
sp["Order Date"]= sp['Order Date'].str.replace('/', '-')
```

In [ ]:
```
sp["Order Date"] = pd.to_datetime(sp["Order Date"], format = '%m-%d-%Y')
```

In [ ]:
```
sp
```

In [ ]:
```
sp['Year']= sp['Order Date'].dt.year
```

In [ ]:
```
sp['Order Date'].dt.month
```

In [ ]:
```
sp['Month']= sp['Order Date'].dt.month_name()
```

In [ ]:
```
sp['Day']= sp['Order Date'].dt.day_name()
```

2. Checking Missing Data

In [ ]:
```
sp.isna().sum().any()
```

In [ ]:
```
any(sp.isnull().sum())
```

In [ ]:
```
sp.isna().any()
```

In [ ]:
```
sp.isnull().sum()  # Count of missing values
```

In [ ]:
```
sp.isna().sum()
```

In [ ]:
```
# Find columns with any null values
sp.columns[sp.isnull().any()]
```

In [ ]:
```
sp[sp['Order ID'].isnull()]
```

In [ ]:
```
# Find rows with any null values
sp[sp.isnull().any(axis=1)]
```

In [ ]:
```
sp.dropna()  # Remove rows with missing values
```

In [ ]:
```
#sp.['Sales'].fillna(0)  # Replace NaNs with 0
```

In [ ]:
```
sp.duplicated().any()
```

In [ ]:
```
sum(sp.duplicated())
```

In [ ]:
```
sp.drop_duplicates()
```

In [ ]:
```
#removing the State column bc it contains just one info
sp.drop(columns = 'State')
```

In [ ]:
```
sp
```

In [ ]:
```
sp = sp.drop('State', axis = 1)
```

3. Unique Values and Frequency Counts

In [ ]:
```
sp['Customer Name'].unique()
```

In [ ]:
```
sp['Customer Name'].nunique()
```

In [ ]:
```
sp['Customer Name'].value_counts()
```

In [ ]:
```
sp['Sub Category'].nunique()
```

In [ ]:
```
sp['Category'].value_counts()
```

In [ ]:
```
sp['Day'].value_counts()
```

In [ ]:
```
for i in sp.columns :
    print(f"{i} : {sp[i].nunique()}")
```

In [ ]:
```
for i in sp.columns:
    print(i, "\n", sp[i].unique())
```

In [ ]:
```
pd.crosstab(sp['Category'], sp['Day'])
```

In [ ]:
```
pd.crosstab(sp['Category'], sp['Day'], margins = True)
```

In [ ]:
```
pd.crosstab(sp['Category'], sp['Day'], margins = True, margins_name='Total')
```

In [ ]:
```
f = pd.crosstab(sp['Month'], sp['Year'], margins = True, margins_name='Total')
print(f)
```

In [ ]:
```
# Convert to percentages
f.div(f.sum(axis=1), axis=0) * 100
```

In [ ]:
```
pd.crosstab(sp['Category'], sp['Day'], normalize=True)
```

In [ ]:
```
f.sort_values(by='Total', ascending=False)
```

4. Basic Operations

In [ ]:
```
sp
```

In [ ]:
```
sp.Profit.mean().round(2)
```

In [ ]:
```
sp['Profit'].mean().round(2)
```

In [ ]:
```
# Assuming 'Profit' is a column in the DataFrame 'sp'
profit_mean = sp['Profit'].mean().round(2)
print(profit_mean)
```

In [ ]:
```
g =sp.Profit.sum()
```

In [ ]:
```
b = sp.Sales.sum()
```

In [ ]:
```
b-g
```

In [ ]:
```
sp.Sales.sum() - sp.Profit.sum()
```

In [ ]:
```
sp
```

In [ ]:
```
sp["Expenses"] = sp.Sales - sp.Profit
```

In [ ]:
```
sp.Profit.sum()/sp.Sales.sum()
```

In [ ]:
```
sp.Profit/sp.Sales
```

In [ ]:
```
sp.Sales.argmax()
```

In [ ]:
```
sp.Sales.idxmax()
```

In [ ]:
```
sp.iloc[267]
```

In [ ]:
```
sp.Sales.max()
```

In [ ]:
```
sp.Profit.argmax()
```

In [ ]:
```
sp.iloc[3159]
```

In [ ]:
```
sp.Profit.max()
```

In [ ]:
```
sp.Discount.sum().round(2)
```

In [ ]:
```
sp.Profit.argmin()
```

In [ ]:
```
sp.iloc[9551]
```

In [ ]:
```
sp.Discount.max()
```

4. Data Selection and Filtering

In [ ]:
```
sp[['Sub Category', 'Category']]  # Select columns
```

In [ ]:
```
sp[sp['Profit'] > 375]  # Rows where 'column' is greater than the mean profit
```

In [ ]:
```
profit_mean = sp.Profit.mean()
```

In [ ]:
```
sp[sp['Profit'] >= profit_mean]
```

In [ ]:
```
sp[sp["Region"]=='North']
```

5. Statistical Analysis

In [ ]:
```
# correlation = strength and direction -1 to + 1 (0.7 to 1)
```

In [ ]:
```
sp[['Sales', 'Profit']].corr()
```

In [ ]:
```
sp[['Sales', 'Profit']].cov()
```

6. Group Analysis: groupby() and Aggregations

In [ ]:
```
cs = sp.groupby('Customer Name')['Profit'].sum().sort_values(ascending = False)
```

In [ ]:
```
cs.head(10)
```

In [ ]:
```
sp.groupby('Customer Name')['Sales'].sum().sort_values(ascending = False)
```

In [ ]:
```
sp.groupby('Customer Name').agg({'Profit':'mean', "Discount":'sum', 'Sales':'sum'})
```

In [ ]:
```
sp.groupby('Category').agg({'Sales': 'sum', 'Profit': 'sum'}).sort_values(by = "Pro
```

In [ ]:
```
sp.groupby('Region')['Sales'].sum().sort_values(ascending = False)
```

In [ ]:
```
sp
```

In [ ]:
```
sp.groupby('Year').agg({'Sales':'sum','Profit':'sum','Discount':'sum'}).sort_values
```

In [ ]:
```
sp.groupby('Month').agg({'Sales':'sum','Profit':'sum','Discount':'sum'}).sort_value
```

In [ ]:
```
sp.groupby('Day').agg({'Sales':'sum','Profit':'sum','Discount':'sum', 'Day':'count'
```

In [ ]:
```
sp.groupby('Month').agg(
    ts = ('Sales', 'sum'),
    tp = ('Profit', 'mean'),
    no = ("Month", 'count')
)
```

In [ ]:
```
 sp.groupby('Day').agg(
    Total_Sales=('Sales', 'sum'),
    Total_Profit=('Profit', 'sum'),
    Total_Discount=('Discount', 'sum'),
    Number_of_Orders=('Day', 'count')
).sort_values(by='Total_Profit', ascending=False)
```

In [ ]:
```
sp.groupby(['Year', 'Month'])['Profit'].sum().sort_values(ascending = False)
```

In [ ]:
```
sp.groupby(['Customer Name', 'Category'])['Profit'].sum()
```

In [ ]:
```
sp.pivot_table(index='Customer Name', columns='Category', values='Profit', aggfunc=
```

In [ ]:
```
sp.pivot_table(index='Category', columns='Region', values='Profit', aggfunc='mean')
```

In [ ]:
```
p = sp.pivot_table(index='Customer Name', columns='Category', values='Sales', aggfu
```

In [ ]:
```
p.sort_values(by='Bakery', ascending=False)
```

In [ ]:
```

```

Visualization

- **Pandas** → Built on Matplotlib, simplifies quick plotting directly from DataFrames.
- **Seaborn** → Built on Matplotlib, simplifies complex statistical visualizations and improves the aesthetics of the plots.
- **Matplotlib** → The foundation of both Pandas and Seaborn visualizations. It provides the core plotting functions, but with more flexibility at the cost of increased complexity.

When to Use Each Library:

- **Pandas:** When you need quick, basic visualizations directly from your DataFrame.
- **Matplotlib:** When you need fine control and customization of plots or when creating more complex visualizations.
- **Seaborn:** When you want aesthetically pleasing plots with less code and when you're focusing on statistical relationships in the data.

In [ ]:
```
sp.head()
```

Most commonly used visuals

- **Bar Chart**: Categorical comparison using rectangular bars.
  - **When to Use**: Comparing different groups or categories (e.g., sales by region, products by popularity).
  - **Use For**: Discrete categories; when you have a few categories.
- **Histogram**: Distribution of a numeric variable using bins.
  - **When to Use**: To observe the frequency distribution of data (e.g., age distribution, income ranges).
  - **Use For**: Understanding the spread or skewness of data.
- **Line Chart**: usually for time-series data.
  - **When to Use**: To show trends or changes over time (e.g., stock prices over months).
  - **Use For**: Tracking changes over a continuous interval (like time).
- **Scatter Plot**: Points plotted on two axes to display the relationship between two variables.

    - **When to Use**: To show correlation or relationships between two numeric variables (e.g., Sales vs. Profit).
    - **Use For**: Identifying relationships, clusters, or outliers.
- **Pie Chart**: Circular chart divided into slices to represent proportions.
    - **When to Use**: To show parts of a whole (e.g., market share, budget allocation).
    - **Use For**: Simple proportional comparisons; avoid with many categories.
- **Boxplot**:Displays distribution, quartiles, and outliers of data.
    - **When to Use**: To compare distributions across categories (e.g., profits across different product lines).
    - **Use For**: Visualizing spread, outliers, and comparing distributions, particularly with statistical data.

In [ ]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

In [ ]:
```
sp.Category.value_counts()
```

In [ ]:
```
sns.countplot(x = "Category", data = sp)
plt.title("Category")
plt.xticks(rotation = 45)
```

In [ ]:
```
sp.Category.value_counts().plot.bar(title="Category Counts")
```

In [ ]:
```
# Bar Plot of Sales per Category - pandas
sp.groupby('Category')['Sales'].sum().sort_values(ascending = False).plot(kind='bar
```

In [ ]:
```
sp.groupby('Category')['Sales'].sum().sort_values(ascending = False)
```

In [ ]:
```
sp.groupby('Category')['Sales'].sum().sort_values(ascending=False).plot(kind='bar',
```

In [ ]:
```
sp.groupby('Year')['Profit'].sum().plot(kind='barh', title='Profit by Year', color=
```

In [ ]:
```
# matplotlib
sc = sp.groupby('Category')['Sales'].sum().sort_values()
print(sc)
```

In [ ]:
```
plt.bar(sc.index, sc.values, color='black')
plt.title('Sales by Category')
plt.ylabel('Total Sales')
plt.xticks(rotation=45)# You can adjust the rotation angle as needed (e.g., 30, 45,
```

In [ ]:
```
y = sp.groupby('Year')['Profit'].sum()
```

In [ ]:
```
y
```

In [ ]:
```
plt.barh(y.index, y.values, color='cyan')
plt.title('Profit by Year')
plt.xlabel('Total Profit')
#plt.yticks(y.index.astype(int)) # Set the y-axis to display only integer values (y
plt.show()
```

In [ ]:
```
# pd & plt
sp.groupby('Category')['Sales'].sum().plot(kind='bar', color='black', title='Sales
plt.ylabel('Total Sales')
```

In [ ]:
```
# seaborn
sns.barplot(x='Region', y='Profit', data=sp, estimator=sum, color='purple') # or pa
#plt.title('Total Sales per Category (Seaborn)')
#plt.xticks(rotation=45)
```

In [ ]:
```
# line chart pandas
sp.groupby('Order Date')['Sales'].sum().plot(kind='line', title='Sales Over Time',
# plt.ylabel('Total Sales')  plt.xlabel('Order Date')
```

In [ ]:
```
# matplotlib
sales_over_time = sp.groupby('Order Date')['Sales'].sum()
plt.plot(sales_over_time.index, sales_over_time.values, color='red')
plt.title('Sales Over Time')
plt.ylabel('Total Sales')
plt.xlabel('Order Date')
plt.xticks(rotation=45)
```

In [ ]:
```
# seaborn
sales_over_time = sp.groupby('Order Date')['Sales'].sum().reset_index()
sns.lineplot(x='Order Date', y='Sales', data=sales_over_time)# color='orange')
plt.title('Sales Over Time')
plt.xticks(rotation=45)
```

In [ ]:
```
# histogram - pandas
sp['Discount'].plot(kind='hist', bins=10, title='Discount Distribution', color='gre
```

In [ ]:
```
# matplotlib
plt.hist(sp['Profit'])#, bins=10, color='green')
plt.title('Discount Distribution')
plt.xlabel('Discount')
```

In [ ]:
```
# seaborn
sns.histplot(sp['Profit'], bins=10, color='green')
plt.title('Discount Distribution')
```

In [ ]:
```
# scatter plot - pandas
sp.plot(kind='scatter', x='Sales', y='Profit', title='Sales vs Profit')
```

In [ ]:
```
# matplotlib
plt.scatter(sp['Sales'], sp['Profit'], color='purple')
```

```
plt.title('Sales vs Profit')
plt.xlabel('Sales')
plt.ylabel('Profit')
```

In [ ]:
```
# Seaborn
sns.scatterplot(x='Sales', y='Profit', data=sp, color='purple')
plt.title('Sales vs Profit')
```

In [ ]:
```
# piechart - pandas
sp.groupby('Region')['Sales'].sum().plot(kind='pie', title='Sales Distribution by R
plt.ylabel('')
```

In [ ]:
```
# matplotlib
sales_by_region = sp.groupby('Region')['Sales'].sum()

plt.pie(sales_by_region, labels=sales_by_region.index, autopct='%1.1f%%')# , starta
plt.title('Sales Distribution by Region')
```

In [ ]:
```
# Box plot - pandas
sp.boxplot(column='Profit', by='Category')
#plt.title('Profit by Category')
#plt.suptitle('')
plt.xticks(rotation=45)
```

In [ ]:
```
# matplotlib
categories = sp['Category'].unique()
p = [sp[sp['Category'] == cat]['Profit'].values for cat in categories]
plt.boxplot(p, labels=categories)
plt.xticks(rotation=45)
```

In [ ]:
```
# seaborn
sns.boxplot(x='Category', y='Profit', data=sp)
plt.title('Profit by Category')
plt.xticks(rotation=45)
```

In [ ]:
```
sp.columns
```

In [ ]:
```
corr = sp[[ 'Sales','Discount', 'Profit']].corr()
print(corr)
```

In [ ]:
```
sns.heatmap(corr)
```

In [ ]:
```
# pip install altair
```

import altair as alt # Bar chart for total sales by category sales_by_category = alt.Chart(sp).mark_bar().encode(x='Category', y='sum(Sales)', color='Category', tooltip=['Category', 'sum(Sales)'] # Tooltip to show category and total sales).properties(title='Total Sales by Category') sales_by_category

In [ ]:

In [ ]:

Outliers

- Outliers are data points that deviate significantly from the rest of the dataset. They can be much higher or lower than most of the values.
- Example: In a dataset of salaries, if most salaries are between $30,000 and $60,000, but one salary is $500,000, this is an outlier.

Why Are Outliers Important?

- **Influence on Results:** Outliers can distort statistical measures (like the mean) and affect machine learning models by leading to biased or inaccurate results.
- **Data Quality:** Outliers might indicate errors in data collection, incorrect data entry, or rare but important cases.

How to Detect Outliers:

- **Boxplots:** Outliers are often represented as points outside the whiskers of a boxplot.
- **Histograms:** Extremely low or high values will appear isolated from the rest of the data.
- **Scatter Plots:** For bivariate data, points that are far from the general cluster can be outliers.
- **IQR (Interquartile Range):** Calculate IQR as the difference between the 75th percentile (Q3) and the 25th percentile (Q1).

Outliers are typically data points that fall below Q1 - 1.5 * IQR or above Q3 + 1.5 * IQR.

Handling Outliers:

- **Keep Them:** In some cases, outliers are valid data points that represent rare but real phenomena.

For example, a company with unusually high sales could represent a valid and valuable case.

- **Remove Them:** If outliers result from errors or noise, removing them can lead to more accurate analysis.
- **Cap Them:** In some cases, you can "cap" outliers by replacing extreme values with maximum or minimum thresholds (known as winsorizing).
- **Transform Data:** Use log transformations or other techniques to reduce the influence of outliers while keeping them in the dataset.

Detecting and Handling Outliers in Python:

- Boxplot Example

In [ ]:
```
sp.boxplot(column='Profit')
```

```
plt.title('Boxplot for Profit')
```

In [ ]:
```
sns.scatterplot(x='Sales', y='Profit', data=sp, color='purple')
```

- IQR Example:

In [ ]:
```
# Calculate Q1 and Q3
Q1 = sp['Profit'].quantile(0.25)
Q3 = sp['Profit'].quantile(0.75)
IQR = Q3 - Q1

print(Q1)
print(Q3)
print(IQR)
```

In [ ]:
```
# Filter outliers
outliers = sp[(sp['Profit'] < (Q1 - 1.5 * IQR)) | (sp['Profit'] > (Q3 + 1.5 * IQR))
```

In [ ]:
```
outliers.shape
```

In [ ]:
```
outliers
```

In [ ]:
```
sp[~(sp['Profit'] < (Q1 - 1.5 * IQR)) | (sp['Profit'] > (Q3 + 1.5 * IQR))]
```

Why and When to Handle Outliers?

- **When Outliers Distort Models:** Machine learning models, especially those based on regression or distance metrics (like KNN), can be highly sensitive to outliers.
- **When Outliers Are Errors:** If outliers are due to data entry mistakes or sensor malfunction, removing or correcting them is important for data integrity.

In [ ]:

In [ ]:
```
sp.head()
```

Pivoting and Unpivoting

In [ ]:
```
pq = pd.read_csv(r"C:\Users\ADMIN\Documents\Excel datsets\PQ Exercise 1d.csv")
```

In [ ]:
```
pq.columns
```

In [ ]:
```
pq.shape
```

In [ ]:
```
pq
```

Unpivot (Melt) the Data

- Transforming the wide format (where dates are columns) into a long format (where dates become rows), using the melt() function.

In [ ]:
```
df = pd.melt(pq, id_vars=['Metric', 'Store', 'Cat'], var_name='Date', value_name='V
# id_vars: These are the columns that will remain as identifiers (in this case, Sto
```

In [ ]:
```
df
```

In [ ]:
```
pd.options.display.max_rows
```

In [ ]:
```
pd.options.display.max_rows = 600
```

Fill Down Missing Values in the Metric Column

- To fill down the Metric column, use ffill() (forward fill).

In [ ]:
```
# Fill down the Metric column
df['Metric'] = df['Metric'].ffill()
```

In [ ]:
```
df.head(5)
```

Pivot the Data (Spread Metric into Sales and Margin)

- Now that the Metric column is filled, you can pivot the data to separate Sales and Margin into different columns.

In [ ]:
```
df = df.pivot_table(index=['Store', 'Cat', 'Date'], columns='Metric', values='Value
```

In [ ]:
```
df.columns.name = None
```

In [ ]:
```
df.isnull().any()
```

In [ ]:
```
df.Date.info()
```

In [ ]:
```
df.sample(10)
```

Convert the Date column to a datetime object

In [ ]:
```
df["Date"] = pd.to_datetime(df['Date'], errors='coerce', format='%m/%d/%Y')
```

In [ ]:
```
df
```

In [ ]:
```
df['Date'].dropna()
```

In [ ]:
```
df.dropna(subset=['Date'])
```

In [ ]:
```
df.dropna(inplace = True)
```

```
In [ ]: pq.head()

In [ ]: df.head(10)

In [ ]: df["Profit"] = round((df["Margin"] * df["Sales"]),2)

In [ ]: df["Year"] = df["Date"].dt.year

In [ ]: df["Month"] = df["Date"].dt.month_name()

In [ ]: pq.head()

In [ ]: df.head()

In [ ]: df["COGS"] = round((df["Sales"] - df["Profit"]),2)

In [ ]: df['Year'] = df['Date'].dt.year

In [ ]: df['Month'] = df['Date'].dt.month_name()

In [ ]: df['Profit'] = round(df['Margin'] * df['Sales'],2)

In [ ]: df['Expenses'] = df['Sales'] - df['Profit']

In [ ]: df.rename(columns= {'Cat': 'Category'}, inplace = True)

In [ ]: df.groupby('Store')['Sales'].sum().sort_values(ascending = False)

In [ ]: df.groupby('Year').agg({'Sales':'sum','Profit':'sum', 'Margin' : 'mean', 'Expenses'

In [ ]: df.groupby('Category').agg({'Sales':'sum','Profit':'sum', 'Margin' : 'mean', 'Expen

In [ ]: df.groupby(['Year', 'Month']).agg({'Sales':'sum','Profit':'sum', 'Margin' : 'mean',

In [ ]:
```